

Project Beacon — Recruiter Rubric & Unipile Data Flow

How the Overall Score is calculated, and exactly how the Unipile-sourced metrics feed into it

1. Overall Score Formula

The Overall Score (0–100) is a weighted composite of the 5 Scored Metrics (Category A). Category B metrics are tracked as signals on the Owner dashboard but carry 0% weight.

OverallScore = $\sum (\text{NormalizedMetricScore}_i \times \text{Weight}_i / 100)$ for $i = 1$ to 5

Normalization logic (applied per metric before weighting):

- Higher-is-better metrics (Outreach Volume, Proactive Sourcing, Progression Rate, Reason-Logged Rate): $\text{NormalizedScore} = \min(100, \text{Actual} \div \text{Good Band Target} \times 100)$
- Lower-is-better metrics (Time-to-First-Touch): $\text{NormalizedScore} = \min(100, \text{Good Band Target} \div \text{Actual} \times 100)$

2. Category A — Scored Metrics (100% of Overall Score)

Metric	Weight	Target	Formula & Logic	DB Table / Columns
Outreach Volume	30%	420 msgs	Count of outbound messages sent by the recruiter this period.	interaction_events COUNT(id) WHERE sender_user_id=recruiterId AND event_type IN ('EMAIL_SENT','LINKEDIN_DM') AND created_at ≥ month start
Proactive Sourcing	30%	34 candidates	Candidates self-sourced by the recruiter, excl. contractor uploads / bulk imports.	leads COUNT(id) WHERE created_by_user_id=recruiterId AND source NOT IN ('CONTRACTOR_SUBMISSION','IMPORT')
Time-to-First-Touch	20%	≤ 1.0 day (lower better)	Avg. business days between lead assignment and recruiter's first outbound message.	leads + interaction_events AVG(BUSINESS_DAYS(lead.claimed_at, first_interaction.created_at))
Progression Rate	10%	80%	% of contacted leads that advance past NEW / CONTACTED stage.	leads + stage_histories (COUNT DISTINCT lead > CONTACTED ÷ TOTAL assigned) × 100
Reason-Logged Rate	10%	100%	% of COLD/DNC leads where the recruiter documented a non-null reason.	lead_flag_events (COUNT reason NOT NULL ÷ COUNT flag_type IN ('DNC','COLD')) × 100

3. Category B — Outcome / Signal Metrics (0% weight, dashboard only)

Metric	Target	Formula & Logic	DB Table / Columns
Onboarding vs. Queue Size	Signal	(Total Onboarded Leads ÷ Total Assigned Leads) × 100	leads COUNT(stage='ONBOARDED') ÷ COUNT(assigned_recruiter_id=recruiterId)
Cold Lead Conversion	18 count	Previously COLD leads revived into an active stage.	stage_histories COUNT WHERE previous_stage='COLD' AND new_stage≠'COLD'
Manual Interviews Conducted	15 count	Recruiter-led manual interview logs outside automated workflows.	manual_activity_logs COUNT WHERE activity_type='INTERVIEW'
Manual Conversion Count	10 count	Direct manual engagements logged to secure high-priority candidates.	manual_activity_logs COUNT WHERE activity_type='DIRECT_ENGAGEMENT'

4. Score Persistence

Snapshots are written to **recruiter_score_snapshots** (overall_score, band_label: Strong / Solid / Coaching / Review, period), **recruiter_metric_snapshots** (per-metric current/previous value, normalized score), and **recruiter_kpi_summaries** (cached aggregates for the Owner dashboard).

5. Where Unipile Fits In: The Underlying Data Flow

Only two Scored Metrics — **Outreach Volume** and **Time-to-First-Touch** — depend on messaging activity, and neither queries Unipile directly at scoring time. **interaction_events** is the Unipile-fed table: Unipile message events (from both a recruiter's LinkedIn account and their email account) are ingested in real time via webhook and written into `interaction_events` as they happen; the rubric formulas simply read from that table afterward. This keeps scoring fast and keeps Unipile a one-way, real-time feed rather than something queried on demand.

6. End-to-End Flow (Webhook → `interaction_events` → Score)

Step	What Happens	Source of Truth
1	Recruiter approves a draft message in Beacon.	Internal app only — no Unipile yet.
2	Beacon calls Unipile's send endpoint. Message enters a rate-limited dispatch queue before actually reaching LinkedIn (pacing rules).	Beacon → Unipile API call.
3	Unipile's New messages webhook fires with event: <code>message_received</code> . Direction resolved via <code>account_info.user_id</code> vs. <code>sender.attendee_provider_id</code> — match = recruiter's own outbound message.	Unipile webhook payload.
4	Webhook handler ACKs within 30s (200 OK), checks the secret header, then writes a row to interaction_events : <code>event_type='LINKEDIN_DM'</code> , <code>sender_user_id=recruiterId</code> , <code>created_at = message's timestamp</code> (provider-ack'd send time — never dispatch-call time).	Unipile timestamp → <code>interaction_events.created_at</code> .
5	De-duplicate using Unipile's <code>message_id</code> so a retried webhook delivery can't create a duplicate row.	Unipile <code>message_id</code> , stored on the row.
6	Outreach Volume = COUNT(<code>interaction_events</code>) WHERE <code>sender_user_id=recruiterId</code> AND <code>event_type</code> IN ('EMAIL_SENT','LINKEDIN_DM') for the month.	Query on <code>interaction_events</code> — no live Unipile call.
7	Time-to-First-Touch = earliest <code>interaction_events</code> row per lead (from Step 4) minus <code>lead.claimed_at</code> , in business days.	<code>leads.claimed_at</code> vs. earliest <code>interaction_events.created_at</code> .

7. One Recruiter, Two Connected Accounts (Email + LinkedIn)

Each recruiter can have **two separate Unipile-connected accounts** — one for their email inbox and one for their LinkedIn account. Unipile treats these as two distinct accounts, each with its own `account_id` and its own provider-side identity, even though both belong to the same recruiter in Beacon. Beacon resolves this to a single recruiter score using one mapping step, not by relying on `sender_id/user_id` alone:

What Unipile Sends	How Beacon Resolves It to a Recruiter
Webhook payload includes <code>account_id</code> (which connected Unipile account received/sent the event) and <code>account_info.user_id</code> (that account's own provider identity — an email address for email, a LinkedIn provider ID for LinkedIn).	Beacon keeps its own mapping table (e.g. <code>recruiter_connected_accounts</code>) linking <code>recruiterId</code> → { <code>email_account_id</code> , <code>linkedin_account_id</code> }. On every webhook event, Beacon looks up <code>account_id</code> against this table to find which recruiter owns it — regardless of channel.
Direction (sent vs. received) is still resolved the same way per channel: compare <code>account_info.user_id</code> to <code>sender.attendee_provider_id</code> .	Once <code>account_id</code> resolves to a <code>recruiterId</code> , that same <code>recruiterId</code> is written to <code>interaction_events.sender_user_id</code> , whether the row came from the email account or the LinkedIn account.
<code>event_type</code> on the stored row reflects the channel: 'EMAIL_SENT' for email, 'LINKEDIN_DM' for LinkedIn.	Outreach Volume counts both <code>event_types</code> together for that <code>recruiterId</code> , so a recruiter's total reflects outreach across both channels, not just one.

In short: **account_id** tells Beacon which channel an event came from, but it's the `account_id` → `recruiterId` mapping — not the channel-specific `user_id` — that decides whose score it counts toward. Beacon never uses `sender_id/user_id` directly as the recruiter identifier, since its format differs per channel (email address vs. LinkedIn provider ID) and can't serve as a stable key on its own.

Note: this rubric version scores 5 weighted metrics (Outreach Volume, Proactive Sourcing, Time-to-First-Touch, Progression Rate, Reason-Logged Rate). Follow-up Persistence, SLA Adherence, and Backlog Aging — covered in the earlier Unipile Metrics Guide as candidate metrics — are not part of the current Category A/B rubric shown above and are not reflected in the Overall Score.